

Date: 01.07.2026

Assignment - 1

UCS3513 System Design Laboratory

Study of System Design Concepts and Development Tools

Name: Tushyent N P
Register No.: 3122 24 5001 189
Class: CSE C

Aim:

- To understand the fundamentals of system design and the architecture of scalable distributed systems.
- To familiarize with the software tools used in the System Design Laboratory and set up the required development environment.

Question:

1. Study the fundamentals of System Design and explore the installation, configuration, and applications of the software tools used in the System Design Laboratory.

Answer:

Fundamentals of System Design

System design is the process of defining the **architecture, components, modules**, interfaces, and data for a system so that it satisfies specified functional and non-functional requirements. It bridges the gap between a problem statement and a working, **scalable** software solution.

- **High-Level Design (HLD):** describes the overall architecture — system components, their interaction, databases used, and technology stack.
- **Low-Level Design (LLD):** describes the internal logic of each component — class diagrams, algorithms, and data structures.
- **Scalability:** the ability of a system to handle increasing load by **scaling up** (adding resources to a single node) or **scaling out** (adding more nodes).
- **Reliability & Availability:** the system continues to function correctly and remain accessible even during partial failures.
- **Distributed Systems:** a collection of independent computers that appear to users as a single coherent system, communicating over a network.

Systems We Will Design This Semester

The laboratory course is structured around eight progressively complex system-design problems, each introducing a core distributed-systems concept:

- **Scalable System Design:** design a system that supports zero to millions of users.
- **Rate Limiter:** design and develop a rate limiter used to control the rate of traffic sent by a client or a service.
- **Consistent Hashing:** design and develop consistent hashing to distribute requests/data efficiently and evenly across servers.

- **Key-Value Store:** design and develop a key-value store to identify a value uniquely by a key.
- **Unique ID Generator:** design and develop a unique ID generator with an auto-increment feature for primary keys in distributed systems.
- **URL Shortener:** design and develop a URL shortener service for shortening a given URL.
- **Web Crawler:** design and develop a web crawler that discovers new or updated content on the web.
- **Search Autocomplete System:** design and develop a system that provides suggestions as the user types in the search box.

Key System Design Concepts & Terminology

The following terms recur throughout the course and form the vocabulary needed to reason about scalable, distributed systems:

Term	Description
Vertical Scaling (Scale Up)	Increasing the capacity of a single server by adding more CPU, RAM, or storage.
Horizontal Scaling (Scale Out)	Increasing capacity by adding more servers to a pool and distributing load across them.
Availability	The percentage of time a system remains operational and accessible, e.g. 99.99% ("four nines").
Reliability	The probability that a system performs its intended function correctly over a given period without failure.
Latency vs Throughput	Latency is the time taken to process one request; throughput is the number of requests handled per unit time.
CAP Theorem	A distributed system can guarantee at most two of Consistency, Availability, and Partition tolerance at the same time.
Load Balancer	Distributes incoming client requests across multiple servers to prevent overload and improve availability.
Caching	Storing frequently accessed data in fast, temporary storage (e.g. Redis) to reduce latency and database load.
Content Delivery Network (CDN)	A geographically distributed set of servers that cache static content closer to users to reduce latency.
Database Replication	Maintaining copies of data on multiple servers (e.g. master-slave) to improve availability and read scalability.
Sharding (Horizontal Partitioning)	Splitting a large database into smaller pieces called shards, distributed across multiple servers, so each server holds only a subset of the data.
Consistent Hashing	A hashing technique that maps servers and keys onto a hash ring so that requests/data are distributed evenly, and only a minimal set of keys need remapping when a node is added or removed.
Rate Limiter	A component that controls the rate of traffic a client or service can send to an API, preventing abuse and ensuring fair usage.

Key-Value Store	A NoSQL database that stores data as pairs of a unique key and its associated value, enabling very fast lookups.
Unique ID Generator	A mechanism (e.g. Twitter Snowflake) that generates globally unique, often time-sortable IDs across multiple distributed servers without relying on a single central counter.
URL Shortener	A service that converts a long URL into a short unique alias and redirects users to the original URL when the alias is accessed.
Web Crawler	An automated program that systematically traverses web pages via hyperlinks to discover new or updated content for indexing.
Search Autocomplete (Typeahead)	A system that suggests likely completions or queries as a user types, commonly implemented using a Trie data structure.
Message Queue	Enables asynchronous communication between services by buffering messages until the receiving service is ready (e.g. Kafka, RabbitMQ).
Reverse Proxy	A server that sits in front of backend servers, handling request routing, SSL termination, and load balancing on their behalf.
Microservices	An architectural style where an application is built as a collection of small, independent, loosely-coupled services communicating over APIs.

Software Tools Used in the System Design Laboratory

The following tools were installed and configured for use in the laboratory:

1. Apache HTTP Server

- **Installation:** downloaded Apache HTTP Server (httpd) and installed it as a local web/proxy server; verified using **httpd -v** in the terminal.
- **Configuration:** edited **httpd.conf** to enable the **mod_proxy** and **mod_proxy_balancer** modules for routing and load balancing.
- **Application:** used as a **reverse proxy / load balancer** that forwards client requests to backend services.

2. Visual Studio Code (VS Code) IDE

- **Installation:** downloaded the installer from code.visualstudio.com and installed it with the "Add to PATH" option enabled.
- **Configuration:** installed the **Extension Pack for Java**, configured the JDK path under Settings, and installed the **GitLens**, **MongoDB** and **Docker** extensions.
- **Application:** used as the primary **IDE** for writing, debugging, and running Java-based microservices.

3. Java (JDK)

- **Installation:** installed JDK 21 (LTS) and set the **JAVA_HOME** environment variable.
- **Application:** provides the runtime and compiler required to build and execute Java applications and services.

4. Apache JMeter

- **Installation:** downloaded the JMeter binary archive, extracted it, and launched it using **jmeter.bat** / **jmeter.sh** (requires JDK to already be installed).
- **Configuration:** created a Thread Group to simulate concurrent users and added HTTP Request samplers pointed at the service endpoints.
- **Application:** used for **load and performance testing** — e.g. verifying that a rate limiter or load balancer behaves correctly under high concurrent traffic.

5. Postman

- **Installation:** installed the desktop client and signed in to sync collections.
- **Application:** used to design, test, and document **REST APIs** exposed by the services being developed.

6. MongoDB

- **Installation:** installed MongoDB Community Server and MongoDB Compass (GUI client); started the service using **mongod**.
- **Configuration:** set the data directory path in **mongod.cfg** and connected to the local instance at **mongodb://localhost:27017**.
- **Application:** a **NoSQL document database** used to persist unstructured/semi-structured data such as key-value records and URL mappings.

7. Apache Kafka

- **Installation:** downloaded Kafka, started ZooKeeper, and then started the Kafka broker using the provided shell scripts.
- **Configuration:** created topics using **kafka-topics.sh --create** and configured the broker's **server.properties** for the local listener.
- **Application:** a distributed **message queue / event-streaming platform** used for asynchronous, decoupled communication between services (e.g. feeding URLs to a web crawler).

8. GitHub

- **Installation:** installed Git from git-scm.com and verified using **git --version**; created a GitHub account and generated an SSH key to link the local machine.
- **Configuration:** set global username and email using **git config --global user.name** and **git config --global user.email**.
- **Application:** used for **version control** and hosting repositories that support collaborative development of distributed applications.

9. Redis

- **Installation:** installed Redis (via WSL/Docker on Windows) and started the server using **redis-server**; verified connectivity with **redis-cli ping**.
- **Configuration:** set the default port (6379) and enabled persistence (RDB snapshotting) in **redis.conf**.
- **Application:** an in-memory **key-value store** used for **caching** and for maintaining counters/tokens in rate-limiter implementations.

Question:

2. Explore the purpose of each software tool used in the System Design Laboratory and prepare a comparison table indicating its role in developing scalable distributed systems.

Answer:

Each tool used in the laboratory plays a distinct role in the **software development lifecycle** of a distributed application — from writing and versioning code to packaging and testing it. The table below summarizes their purpose and their specific contribution to building scalable distributed systems.

Tool	Category	Role in Scalable Distributed Systems
Apache HTTP Server	Reverse Proxy / Load Balancer	Routes and balances incoming client requests across multiple backend server instances.
VS Code IDE	IDE	Provides an integrated environment for writing, debugging, and refactoring code with extension support for multiple languages.
Java (JDK)	Runtime / Compiler	Compiles and executes Java code, forming the base runtime for backend services.
JMeter	Load / Performance Testing	Simulates concurrent users to measure throughput and latency, validating scalability under load.
Postman	API Testing	Validates and documents the REST APIs through which distributed services communicate.
MongoDB	NoSQL Database	Stores unstructured/semi-structured data (e.g. key-value records, URL mappings) with horizontal scalability via sharding.
Kafka	Message Queue / Event Streaming	Enables asynchronous, decoupled communication between services, buffering high-throughput event streams.
GitHub	Version Control / Repository Hosting	Hosts remote repositories, enables code review via pull requests, and automates build/test pipelines using Actions.
Redis	In-Memory Key-Value Store	Provides fast caching and counter/token storage, commonly used to implement rate limiters.

Together, these tools cover the complete workflow: **Apache** for request routing, **VS Code and Java** for development, **JMeter and Postman** for testing, **MongoDB and Redis** for data storage/caching, **Kafka** for asynchronous messaging, and **GitHub** for version control — all of which are essential to reliably building and scaling a distributed system.

Question:

3. Create a GitHub repository, configure a Java project in Visual Studio Code, and upload the project to GitHub.

Answer:

Step 1: Create a GitHub Repository

- Logged in to GitHub and clicked **New Repository**.
- Provided a repository name, an optional description, set visibility to **Public**, and initialized it with a **README.md** and a Java **.gitignore** file.
- Clicked **Create repository** to generate the remote repository.

Step 2: Configure a Java Project in VS Code

- Cloned the newly created repository locally using **git clone <repository-url>** in the VS Code integrated terminal.
- Opened the cloned folder in VS Code and used the **Java: Create Java Project** command from the Command Palette to generate the standard **src / bin** structure.
- Wrote a sample **Main.java** class and verified it compiled and ran correctly using the built-in Java Run/Debug option.

Step 3: Upload the Project to GitHub

- Staged all changes: **git add ..**
- Committed the changes with a message: **git commit -m "Initial commit: Java project setup"**.
- Pushed the commit to the remote repository: **git push origin main**.
- Verified on the GitHub website that the project files, including the source code, were successfully uploaded.

GitHub Repository Link: <https://github.com/Tushyent/system-design-lab-demo>

Learning Outcomes:

- I have learned the fundamental concepts of system design, including high-level and low-level design and the principles of scalable distributed systems.
- I have understood the purpose and role of each software tool — Apache, VS Code, Java, JMeter, Postman, MongoDB, Kafka, GitHub, and Redis — used in the laboratory.
- I have implemented a complete workflow by creating a GitHub repository, configuring a Java project in VS Code, and uploading it to GitHub.